What is this?

This notebook is part of a collection of `pluto` notebooks on various topics discussed during the Time Domain Astrophysics course delivered by Stefano Covino at the Università dell'Insubria in Como (Italy). Please direct questions and suggestions to stefano.covino@inaf.it.

Table of Contents

Modeling and forecasting atmospheric CO_2
- Carbon Dioxide modeling and forecast
- Reference & Material
  - Credits
  - Course Flow

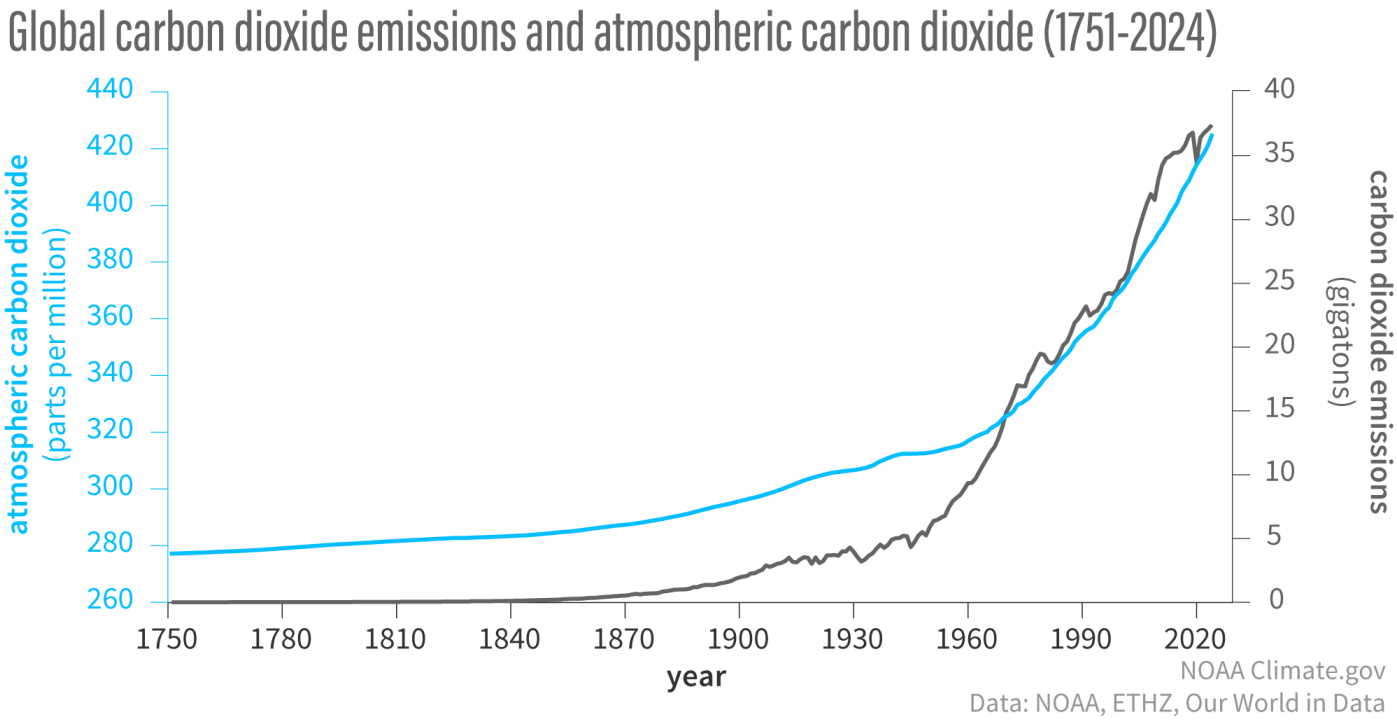
Modeling and forecasting atmospheric CO

2

- Each year, human activities release more carbon dioxide into the atmosphere than natural processes can remove, causing the amount of carbon dioxide in the atmosphere to increase.
- The global average carbon dioxide set a new record every year, we are now at more than 400 parts per million ("ppm").
- Atmospheric carbon dioxide is now 50% higher than it was before the Industrial Revolution.
- The ocean has absorbed enough carbon dioxide to lower its pH by 0.1 units, a 30% increase in acidity.

- The years with the largest annual carbon dioxide growth tend to be associated with the strongest El Niños —the warm phase of a natural climate pattern in the tropical Pacific— which lead to high temperatures over land and sea and an expansion of global drought area.
- In turn, these weather conditions typically lead to less plant growth, which reduces carbon dioxide uptake, as well as increased decomposition of carbon in soil and increased carbon dioxide emissions from forest fires.
- Together, these impacts cause atmospheric carbon dioxide levels to rise faster than normal.

- Carbon dioxide concentrations are rising mostly because of the fossil fuels that people have been burning for energy since the start of the Industrial Revolution. Fossil fuels like coal and oil contain the carbon from millions of years of photosynthesis. We are returning that carbon to the atmosphere in just a few hundred years. Since the middle of the 20th century, annual emissions of carbon dioxide from burning fossil fuels have increased every decade, from close to 11 billion tons of carbon dioxide per year in the 1960s to about 40 billion tons in 2024.



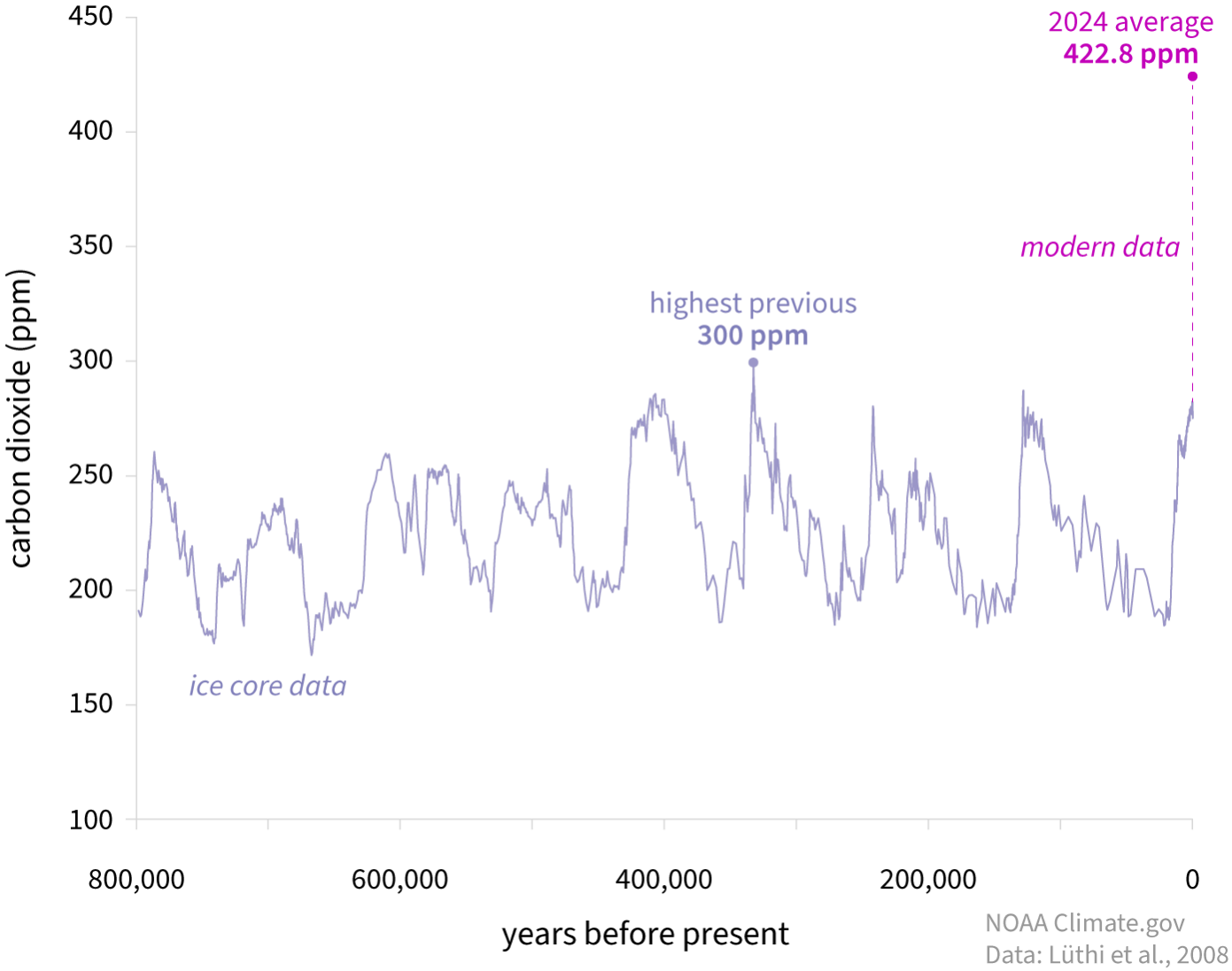
- The amount of carbon dioxide in the atmosphere (blue line) has increased along with human emissions (gray line) since the start of the Industrial Revolution in 1750.

- Natural "sinks," including plant growth and ocean absorption, remove about half of carbon dioxide humans emit into the atmosphere from fossil fuel burning. The rest stays in the atmosphere. Every year, we are adding more carbon dioxide to the atmosphere than natural sinks can remove,

and so the total amount of carbon dioxide in the atmosphere goes up. The bigger the overshoot, the faster atmospheric carbon dioxide increases.
- In the 1960s, atmospheric carbon dioxide increased by about 0.8 ppm per year on average. That growth rate doubled to an average of 1.6 ppm per year in the 1980s and 1.5 ppm per year in the 1990s. In the last decade (2015-2024), the annual increase has accelerated to 2.6 ppm per year. As a result, the annual rate of increase in atmospheric carbon dioxide over the past 60 years is 100-200 times faster than the increase that occurred at the end of the last ice age 11,000-17,000 years ago.

- Carbon dioxide is Earth's most important long-lived greenhouse gas: a gas that absorbs and radiates heat. Unlike oxygen or nitrogen, greenhouse gases absorb heat radiating from the Earth's surface and re-release it in all directions. Without any carbon dioxide, Earth's natural greenhouse effect would be too weak to keep the average global surface temperature above freezing.
- By adding more carbon dioxide to the atmosphere, we are amplifying the natural greenhouse effect, causing global temperature to rise. According to observations and analysis by the NOAA Global Monitoring Laboratory, carbon dioxide alone is responsible for about 80% of the total heating influence of all human-produced greenhouse gases since 1990.
- Carbon dioxide also dissolves into the ocean reacting with water molecules, producing carbonic acid and lowering the ocean's pH (raising its acidity). Since the start of the Industrial Revolution, the pH of the ocean's surface waters has dropped from 8.21 to 8.10. This drop in pH is called ocean acidification, and it interferes with the ability of marine life to extract calcium from sea water to build skeletons and shells.

- Natural increases in carbon dioxide concentrations have periodically warmed Earth's temperature during ice age cycles over the past million years or more.
- The warm episodes (interglacials) began with a small increase in incoming sunlight in the Northern Hemisphere summer due to variations in Earth's orbit around the Sun and its axis of rotation.
- The little increase in sunlight caused a small warming. Yet, as the oceans warmed, they outgassed carbon dioxide. The extra carbon dioxide in the atmosphere greatly amplified the initial, solar-driven warming.
- Based on air bubbles trapped in mile-thick ice cores and other paleoclimate evidence, we know that during the ice age cycles of the past million years or so, atmospheric carbon dioxide didn't get any higher than 300 ppm. Before the Industrial Revolution started in the mid-1700s, atmospheric carbon dioxide was 280 ppm or less.



- Atmospheric carbon dioxide (CO_2) in parts per million (ppm) for the past 800,000 years based on ice-core data (light purple line) compared to 2024 concentrations (bright magenta dot). The peaks and valleys in the line show ice ages (low CO_2) and warmer interglacials (higher CO_2). Throughout that time, CO_2 was never higher than 300 ppm (light purple dot, between 300,000 and 400,000 years ago).

- Carbon dioxide levels today are higher than at any point in human history. In fact, the last time atmospheric carbon dioxide amounts were this high was roughly 3 million years ago, during the Mid-Pliocene Warm Period, when global surface temperature was 2.5–4 degrees Celsius warmer than during the pre-industrial era. Sea level was at least 5m higher than it was in 1900 and possibly as much as 25m higher.

Carbon Dioxide modeling and forecast

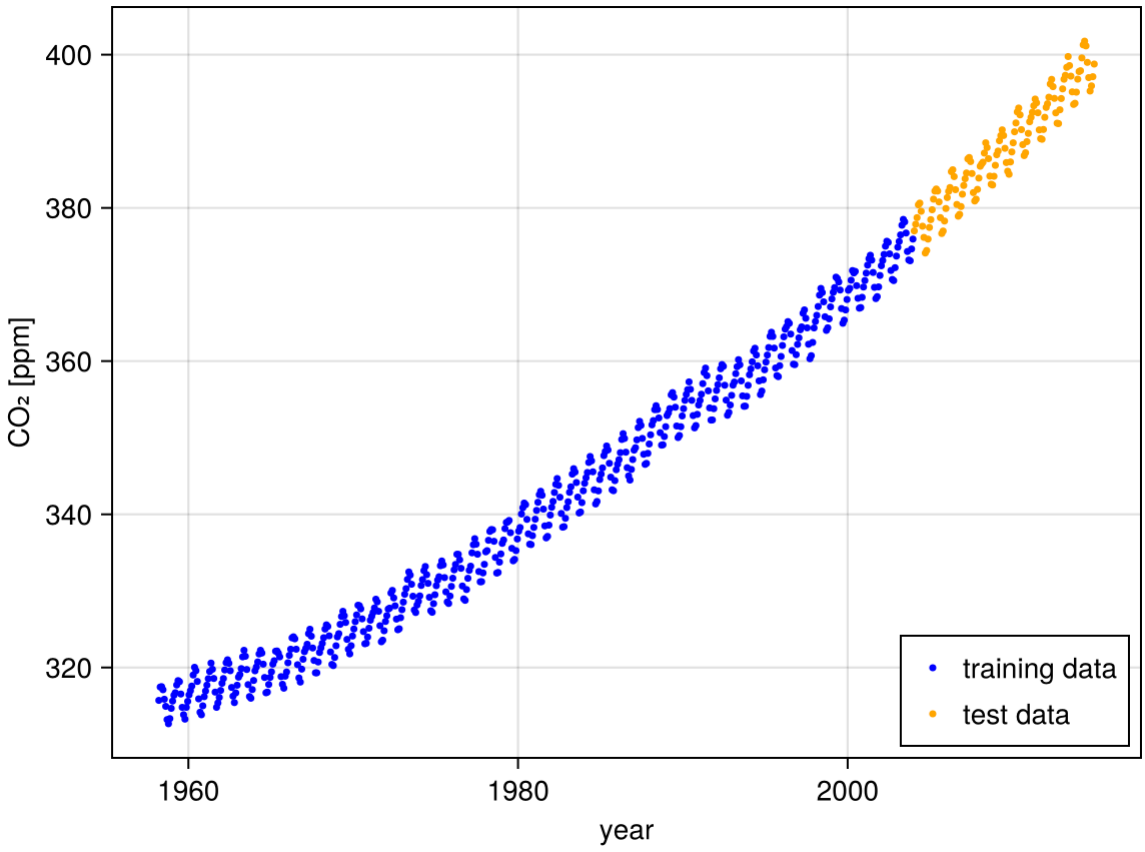
- We now apply Gaussian process regression to the Mauna Loa CO_2 dataset. This exercise might require a farly advanced coding skill, yet it is anyway of interest since it showcases a rich combination of kernels, and how to handle and optimize all their parameters.
- First, let's load the data, and extract the variable of our interest. We separate data betwwen a training and test set, as it is common in machine learning.

```julia
(xtrain, ytrain), (xtest, ytest) = let
    data = CSV.read("CO2_data.csv", Tables.matrix; header=0)
    year = data[:, 1]
    co2 = data[:, 2]

    # We split the data into training and testing set:
    idx_train = year .< 2004
    idx_test = .!idx_train

    (year[idx_train], co2[idx_train]), (year[idx_test], co2[idx_test])  # block's return value
end;
```

- And plot the dataset:



- The time-series is quite impressive. We see an increase with a steepening in the recent years with superposed a seasonal variation.
- We will model this dataset using a sum of several kernels which describe:
  - smooth trend: squared exponential kernel with long lengthscale;
  - seasonal component: periodic covariance function with period of one year, multiplied with a squared exponential kernel to allow decay away from exact periodicity;
  - medium-term irregularities: rational quadratic kernel;
  - noise terms: squared exponential kernel with short lengthscale and uncorrelated observation noise.
- For more details, see Rasmussen & Williams (2005), chapter 5.

- We first define the parameters of the covariance functions we are going to pliug into our GP:

```
θ_init = (;
    se_long = (;
        σ = positive(exp(4.0)),
        ℓ = positive(exp(4.0)),
    ),
    seasonal = (;
        # product kernels only need a single overall signal variance
        per = (;
            ℓ = positive(exp(0.0)),  # relative to period!
            p = fixed(1.0),  # 1 year, do not optimize over
        ),
        se = (;
            σ = positive(exp(1.0)),
            ℓ = positive(exp(4.0)),
        ),
    ),
    rq = (;
        σ = positive(exp(0.0)),
        ℓ = positive(exp(0.0)),
        α = positive(exp(-1.0)),
    ),
    se_short = (;
        σ = positive(exp(-2.0)),
        ℓ = positive(exp(-2.0)),
    ),
    noise_scale = positive(exp(-2.0)),
);
```

- Then, let's begin to build the GP priors (our kernel):

```
RQ (generic function with 1 method)
```

```
begin
    SE(θ) = θ.σ^2 * with_lengthscale(SqExponentialKernel(), θ.ℓ)
    Per(θ) = with_lengthscale(PeriodicKernel(; r=[θ.ℓ / 2]), θ.p)
    RQ(θ) = θ.σ^2 * with_lengthscale(RationalQuadraticKernel(; α=θ.α), θ.ℓ);
end
```

- This allows us to write a function that, given the nested tuple of parameter values, constructs the GP prior:

```
build_gp_prior (generic function with 1 method)
```

```
function build_gp_prior(θ)
    k_smooth_trend = SE(θ.se_long)
    k_seasonality = Per(θ.seasonal.per) * SE(θ.seasonal.se)
    k_medium_term_irregularities = RQ(θ.rq)
    k_noise_terms = SE(θ.se_short) + θ.noise_scale^2 * WhiteKernel()
    kernel = k_smooth_trend + k_seasonality + k_medium_term_irregularities +
k_noise_terms
    return GP(kernel)  # [`ZeroMean`](@ref) mean function by default
end
```

- To construct the posterior, we need to first build a FiniteGP, which represents the infinite-dimensional GP at a finite number of input features:

```
build_finite_gp (generic function with 1 method)
```

```
function build_finite_gp(θ)
    f = build_gp_prior(θ)
    return f(xtrain)
end
```

- We obtain the posterior, conditioned on the (finite) observations, by calling posterior:

```
build_posterior_gp (generic function with 1 method)
```

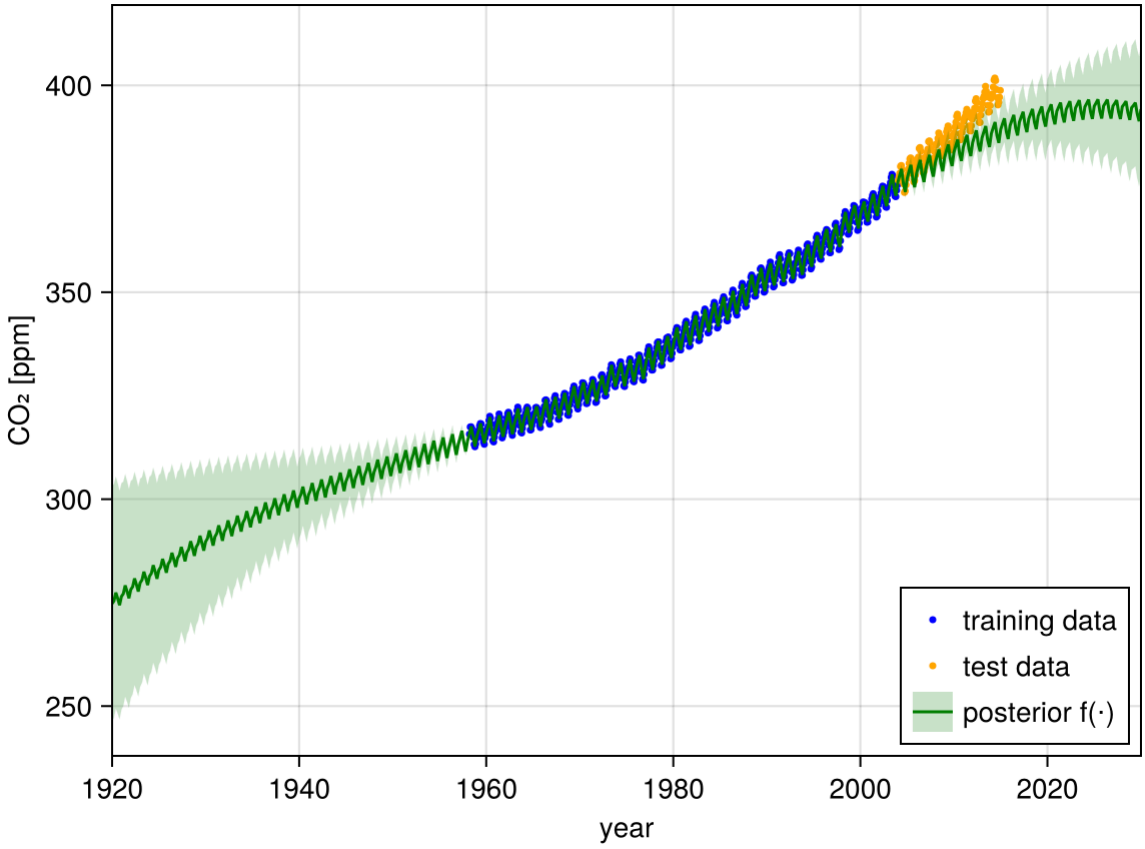
```
function build_posterior_gp(θ)
    fx = build_finite_gp(θ)
    return posterior(fx, ytrain)
end
```

- Now we can put it all together to obtain a PosteriorGP. The call to ParameterHandling.value is required to replace the constraints (such as positive in our case) with concrete numbers:
- Now we can put it all together to obtain a PosteriorGP:

```
fpost_init =
  PosteriorGP(GP(ZeroMean(), Sum of 5 kernels:
                             Squared Exponential Kernel (metric = Distances.Euclidean(0.0))
```

```
fpost_init = build_posterior_gp(ParameterHandling.value(θ_init))
```

- Let's visualize what the GP fitted to the data looks like, for the initial choice of kernel hyperparameters.



- The “first guess” parameters are not bad, yet we definitely need to optimize the hyperparameter of the model.
- To improve the fit, we want to maximize the (log) marginal likelihood with respect to the hyperparameters, so we define a loss function to minimize.

```
loss (generic function with 1 method)
```

```
function loss(θ)
    fx = build_finite_gp(θ)
    lml = logpdf(fx, ytrain)  # this computes the log marginal likelihood
    return -lml
end
```

- The L-BFGS algorithm was chosen because it seems to work effectively.

```
optimize_loss (generic function with 1 method)
```

```
begin
    default_optimizer = LBFGS(;
        alphaguess=Optim.LineSearches.InitialStatic(; scaled=true),
        linesearch=Optim.LineSearches.BackTracking(),
    )

    function optimize_loss(loss, θ_init; optimizer=default_optimizer, maxiter=1_000)
        options = Optim.Options(; iterations=maxiter, show_trace=true)

        θ_flat_init, unflatten = ParameterHandling.value_flatten(θ_init)
        loss_packed = loss ∘ unflatten

        #
        function fg!(F, G, x)
            if F !== nothing && G !== nothing
                val, grad = Zygote.withgradient(loss_packed, x)
                G .= only(grad)
                return val
            elseif G !== nothing
                grad = Zygote.gradient(loss_packed, x)
                G .= only(grad)
                return nothing
            elseif F !== nothing
                return loss_packed(x)
            end
        end

        result = optimize(NLSolversBase.only_fg!(fg!), θ_flat_init, optimizer, options; inplace=false)

        return unflatten(result.minimizer), result
    end
end
```

- We now run the optimization:

```
((se_long = (σ = 437.147, ℓ = 154.784), seasonal = (per = (ℓ = 1.45577, p = 1.0), se = (σ = 2.44
```

```
θ_opt, opt_result = optimize_loss(loss, θ_init)
```

```
Iter     Function value   Gradient norm
     0     2.285662e+02     3.466984e+02
 * time: 0.01501011848449707
     1     1.597968e+02     5.811062e+01
 * time: 0.8566560745239258
     2     1.516902e+02     4.934826e+01
 * time: 1.5630319118499756
     3     1.311976e+02     4.436591e+01
 * time: 2.3622710704803467
     4     1.250485e+02     2.431021e+01
 * time: 2.5763449668884277
     5     1.208493e+02     7.951602e+00
 * time: 3.3270089626312256
     6     1.190798e+02     9.556037e+00
 * time: 3.5897610187530518
     7     1.172102e+02     7.308274e+00
 * time: 4.220891952514648
     8     1.161961e+02     5.340282e+00
 * time: 4.367722988128662
     9     1.157434e+02     2.651421e+00
 * time: 5.088284969329834
    10     1.156009e+02     1.236177e+00
 * time: 5.268074989318848
    11     1.155254e+02     7.740342e-01
 * time: 5.46354603767395
    12     1.154964e+02     3.221118e-01
 * time: 6.102132081985474
    13     1.154896e+02     1.039792e+00
 * time: 6.248834133148193
    14     1.154860e+02     1.202208e+00
 * time: 6.994910001754761
    15     1.154829e+02     3.328780e-01
 * time: 7.225353956222534
    16     1.154811e+02     2.493976e-01
 * time: 7.448671102523804
    17     1.154783e+02     5.207502e-01
```

- Let's construct the posterior GP with the optimized hyperparameters:

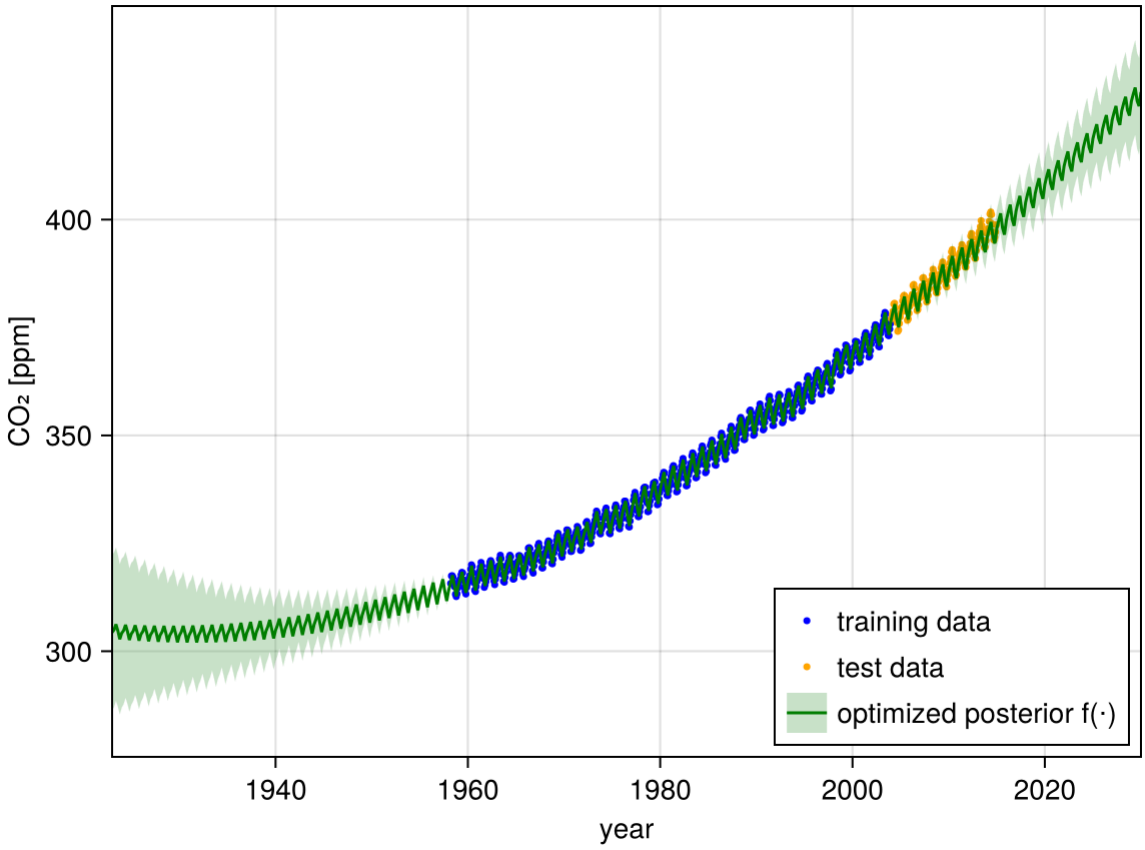
```
fpost_opt =
  PosteriorGP(GP(ZeroMean(), Sum of 5 kernels:
                             Squared Exponential Kernel (metric = Distances.Euclidean(0.0))
```

```
fpost_opt = build_posterior_gp(ParameterHandling.value(θ_opt))
```

- And, finally, we can visualize our optimized posterior GP:

Plot lower limit:

Year: 1923



- Now the fit is, at least visually, satisfactorarily. And one can appreciate the expected increase in CO_2 content in atmosphere after the last measrements in our dataset.
  - Actually, test data seem to show an even more rapid increase than the extrapolation based upon the training data.
- It is possible that in the future the model has to be upgraded to take care of a new behavior, likely bads news for the planet, anyway.

Reference & Material

Material and papers related to the topics discussed in this lecture.

- Ma et al. (2021) - Can Machine Learning be Applied to Carbon Emissions Analysis: An Application to the CO_2 Emissions Analysis Using Gaussian Process Regression
- Tian et al. (2025) - Carbon Dioxide Emission Forecast: A Review of Existing Models and Future Challenges

Credits

This notebook contains material obtained from https://juliagaussianprocesses.github.io/AbstractGPs.jl/dev/examples/1-mauna-loa/.

Course Flow

	Previous lecture	Next lecture	Course Summary
notebook	Lecture about Gaussian processes	Lecture about AI interaction	Course Summary
html	Lecture about Gaussian processes	Lecture about AI interaction	Course Summary

Notebook v1.0.1 - 17 May 2026